MedShield-FL

A Unified Privacy-Preserving Federated Learning Framework
with Homomorphic Encryption for Secure Healthcare AI

Complete Project Build Guide — 100 Tasks with Acceptance Criteria

---

“Train accurate medical AI across hospitals — without patient data ever leaving the hospital.”

Domain: Federated Learning · Homomorphic Encryption · Vision Transformers · Explainable Healthcare AI

Reference dataset: BraTS Brain-MRI | Model: ViT-Base/16 | Encryption: CKKS (TenSEAL) | FL: Flower (FedAvg)

Structured for step-by-step build by an AI coding agent and a developer.
Each task ends with 3–4 checkable acceptance criteria.

Table of Contents

Part A · Project Overview

A.1 Introduction — the problem in plain language

Hospitals collect enormous amounts of medical data every day — X-rays, MRI scans, patient records. This data could train Artificial Intelligence models that help doctors detect diseases like brain tumors faster and more accurately. But privacy laws such as **HIPAA** (USA) and **GDPR** (Europe) forbid hospitals from freely sharing patient data. As a result, no single hospital has enough varied data to train a truly reliable model — a model trained on one hospital tends to fail on patients or scanners it has never seen.

MedShield-FL solves this by moving the model to the data, not the data to the model. Every hospital trains the shared AI model on its own private data. Only the *learnings* (model updates) are shared — and even those are locked with strong **homomorphic encryption** before leaving the hospital, so not even the central server can ever see the real values. Many hospitals can thus build one powerful shared model while every patient record stays exactly where it is: inside its own hospital.

A.2 Problem Statement

Teams building medical AI face a hard contradiction: models need large, varied datasets to be accurate, yet hospitals hold data in small isolated pockets and privacy laws prohibit centralising it. Existing fixes each fall short — basic differential privacy hurts accuracy, encrypting everything is too slow and bandwidth-heavy, expert labelling time is scarce and expensive, and doctors distrust black-box AI they cannot interpret. MedShield-FL is designed to solve all of these together in one unified framework, using Brain-MRI tumor detection on the BraTS dataset as its concrete target.

A.3 Objectives

- **Collaborative training:** let multiple hospitals jointly train one shared model without moving raw patient data.
- **Mathematical privacy:** use Homomorphic Encryption (CKKS) so the server only ever touches encrypted values.
- **Lower bandwidth:** use Vision Transformers and encrypt only the most critical parts (e.g. the [CLS] token) — cutting transfer cost dramatically.
- **Smart use of doctors' time:** use Active Learning to surface only the most uncertain, useful images for labelling.
- **Handle real-world differences:** stay accurate across hospitals with different scanners and populations (domain shift).
- **Clinical trust:** use Explainable AI (Grad-CAM) so doctors see a heatmap of what drove each decision.

A.4 How the System Works (one training round)

- **1. Start:** the central server sends the current shared model to every hospital.
- **2. Local training:** each hospital trains it on its own images; data never leaves the premises.
- **3. Lock the results:** the hospital encrypts its update into unreadable ciphertext.
- **4. Send:** only the encrypted update is sent — never raw data, never even the plain update.

- **5. Blind averaging:** the server averages all encrypted updates without ever unlocking them.
- **6. Return:** the new (still-encrypted) combined model goes back to every hospital.
- **7. Unlock locally:** each hospital decrypts its copy with its own private key, which never leaves the hospital.
- **8. Repeat:** the cycle repeats; with every round the shared model gets smarter, privacy intact.

A.5 System Architecture

The system has two sides talking over the internet:

Central Server side

- **Orchestration Hub** — manages connected hospitals and keeps every round in sync.
- **Encrypted Aggregator** — combines locked updates using homomorphic math, never unlocking them.
- **Global Model Store** — safely holds the current (encrypted) shared model.

Hospital Client side

- **Data Preparation Module** — cleans and prepares local MRI images.
- **Smart Labelling Assistant** — flags only the most useful, uncertain images for a doctor.
- **Local AI Training Engine** — trains the shared model on the hospital's own data.
- **Encryption Manager** — locks updates before they leave and unlocks incoming updates; the private key never leaves.

Privacy by design: the frontend never touches raw patient data or private keys — all sensitive work (training, encrypting, aggregating) happens on the backend inside each hospital's secured environment.

A.6 Technology Stack

Layer	Technology	Role in MedShield-FL
Frontend	React.js, TypeScript, Tailwind CSS	Clean dashboard for hospital staff, admins and doctors (display only).
Backend API	FastAPI (Python)	Serves the app, auth, predictions, labelling and metrics.
AI Framework	PyTorch — ViT-Base/16	Vision Transformer that classifies brain-tumor MRI slices.
Federated Learning	Flower (FedAvg)	Coordinates rounds and aggregates hospital updates.
Encryption	TenSEAL — CKKS scheme	Homomorphic encryption so the server never sees plaintext updates.
Database	PostgreSQL	Stores hospitals, rounds, model versions, predictions, audit logs.
Explainable AI	Grad-CAM	Heatmaps showing which region drove each prediction.
Dataset	BraTS Brain-MRI	Multi-hospital, expert-labelled MRI benchmark (T1, T1ce, T2, FLAIR).

A.7 Key Features & Innovations

- One unified framework instead of disconnected privacy tools.
- Selective (token-level) homomorphic encryption for major bandwidth savings.
- Robustness to domain shift across different hospitals and scanners.
- Active learning to minimise expensive expert labelling.
- Built-in explainability (Grad-CAM) for clinical trust.
- Future scope: zero-knowledge proofs against bad actors, differential privacy layer, GPU-accelerated encryption, edge deployment, and more disease/scan types.

Part B · Prerequisites

Before starting the 100 tasks, the following should be in place. These are the assumptions every task builds on.

B.1 Knowledge prerequisites

- **Python & PyTorch:** training neural networks, tensors, datasets and dataloaders.
- **Deep learning basics:** image classification, overfitting, evaluation metrics, transfer learning.
- **Vision Transformers:** the idea of patches, tokens and the [CLS] token (conceptual is enough to start).
- **Federated Learning:** the client/server round model and FedAvg (the Flower docs cover this).
- **Homomorphic encryption (conceptual):** that CKKS lets you add/multiply on encrypted numbers without decrypting.
- **Web development:** React + TypeScript, REST APIs (FastAPI), and basic SQL/PostgreSQL.

B.2 Software & tools

- Python 3.10+ and pip/venv; Node.js 18+ and npm; Git.
- PyTorch, timm (for ViT), Flower (flwr), TenSEAL, FastAPI, Uvicorn, SQLAlchemy, Alembic.
- PostgreSQL 14+ (Docker recommended); Docker & Docker Compose.
- React + TypeScript (Vite), Tailwind CSS, a component library (e.g. shadcn/ui).
- A code editor and, for this project's purpose, an AI coding agent to execute tasks.

B.3 Hardware

- A GPU (NVIDIA, 8 GB+ VRAM) is strongly recommended for ViT training; CPU works for small tests but is slow.
- 16 GB+ system RAM (homomorphic encryption is memory-heavy).
- Enough disk for the BraTS dataset plus model checkpoints (tens of GB).

B.4 Data & accounts

- Legitimate access to the BraTS Brain-MRI dataset (respect its licence and usage terms).
- For simulation, the dataset will be split into several 'hospital' partitions on one or more machines.
- Optional: an experiment-tracking account (e.g. Weights & Biases) or local TensorBoard.
- No real patient data or PHI is required — BraTS is a de-identified research benchmark.

Part C · How to Use This Document

This document breaks MedShield-FL into **100 tasks across 10 levels (Level 0–9)**. The levels are ordered by dependency: each level builds on the ones before it, so working top to bottom keeps the project always in a runnable state. It is written to be handed to an AI coding agent and followed by a developer together.

How each task is structured

- **Task number & title** — one clear, self-contained unit of work.
- **What to build** — a short plain-language description of the goal of the task.
- **Acceptance criteria** — 3–4 checkable statements (marked ☐). The task is **done only when every box is true**. These are written so both an AI agent and a human reviewer can objectively confirm completion.

Suggested workflow

- Do tasks in order within a level; levels can slightly overlap once their dependencies exist.
- Before marking a task complete, tick every acceptance criterion — treat unticked boxes as unfinished work.
- Commit after each task; keep the acceptance criteria in the pull-request description as the review checklist.
- When feeding a task to an AI agent, paste the task title, description, and its acceptance criteria as the definition of done.

Levels 0–4 build the privacy-preserving core (setup, data, model, encryption, federated learning). Levels 5–6 add active learning and explainability. Levels 7–8 add the backend API and the dashboard. Level 9 integrates, tests, secures, and deploys the whole system.

Part D · The 100 Tasks

LEVEL 0 — FOUNDATIONS & ENVIRONMENT SETUP

Tasks 1–8. Stand up the repository, toolchain, and services every later level depends on. Nothing here trains a model — it makes the rest of the build reproducible and runnable by any team member or AI agent.

Task 1 — Create the project repository and folder structure

Initialise a single Git monorepo with clearly separated areas for the FastAPI backend, the Flower federated-learning code, the React frontend, shared utilities, data, and documentation.

Acceptance criteria (done when all are true):

- ☐ A git repository exists with top-level folders (e.g. /server, /fl, /frontend, /shared, /data, /docs) and a root README describing each.
- ☐ A .gitignore excludes virtual environments, node_modules, datasets, model checkpoints, and .env files.
- ☐ The repository clones cleanly on a fresh machine with no missing-folder errors.
- ☐ A LICENSE and a CONTRIBUTING/setup note are present at the root.

Task 2 — Set up the Python backend environment

Create a reproducible Python 3.10+ environment with all core backend and ML dependencies pinned.

Acceptance criteria (done when all are true):

- ☐ A virtual environment is created and a requirements.txt (or pyproject.toml) pins FastAPI, Uvicorn, PyTorch, Flower, TenSEAL, timm, SQLAlchemy, and Alembic with explicit versions.
- ☐ `pip install` completes on a clean environment with zero dependency conflicts.
- ☐ `python -c "import torch, flwr, tenseal, fastapi"` runs without ImportError.
- ☐ A short script prints the versions of all four core libraries to confirm the environment is active.

Task 3 — Set up the frontend environment

Scaffold a React + TypeScript app with Tailwind CSS and a component library, ready for the dashboard.

Acceptance criteria (done when all are true):

- ☐ A React + TypeScript project (Vite) builds and serves a default page with `npm run dev`.
- ☐ Tailwind CSS is configured and a test utility class visibly changes styling.
- ☐ A component library (e.g. shadcn/ui) is installed and one sample component renders.
- ☐ `npm run build` produces a production bundle with no TypeScript errors.

Task 4 — Provision the PostgreSQL database

Run PostgreSQL locally (via Docker) and confirm the backend can connect to it.

Acceptance criteria (done when all are true):

- ☐ A PostgreSQL instance starts via docker-compose and is reachable on its configured port.
- ☐ A dedicated database and application user are created with correct privileges.
- ☐ The backend connects using a connection string read from configuration and runs a `SELECT 1` health check.
- ☐ Connection credentials are read from environment variables, not hard-coded.

Task 5 — Configure environment variables and secrets

Centralise all configuration (DB URL, JWT secret, FL server address, encryption params) in a typed settings module driven by environment variables.

Acceptance criteria (done when all are true):

- ☐ A `.env.example` lists every required variable with safe placeholder values.
- ☐ A settings/config module loads and validates variables at startup and fails fast with a clear message if any required value is missing.
- ☐ No secret or credential appears anywhere in committed source code.
- ☐ Switching an environment variable (e.g. DB host) changes app behaviour without any code edit.

Task 6 — Set up code-quality tooling

Add linting, formatting, and pre-commit hooks for both Python and TypeScript so all contributors (human or AI) produce consistent code.

Acceptance criteria (done when all are true):

- ☐ Python formatting/linting (e.g. black + ruff) and TS tooling (eslint + prettier) are configured and runnable via a single command each.
- ☐ A pre-commit configuration runs the linters/formatters automatically before each commit.
- ☐ Running the format command on the whole repo leaves it in a stable, no-further-changes state.
- ☐ Linting passes on the existing codebase with zero errors.

Task 7 — Containerise all services

Write Dockerfiles and a docker-compose file that bring up backend, frontend, database, and FL server together.

Acceptance criteria (done when all are true):

- ☐ A `docker-compose up` command starts backend, frontend, PostgreSQL, and the FL server without manual steps.
- ☐ Each service has its own Dockerfile that builds successfully from a clean cache.
- ☐ Services can reach each other by their compose service names (verified by a health call).
- ☐ Stopping and restarting the stack preserves database data via a named volume.

Task 8 — Set up version control workflow and CI

Define a branching workflow and a Continuous Integration pipeline that lints and runs tests on every push.

Acceptance criteria (done when all are true):

- ☐ A documented branch/PR workflow exists (e.g. feature branches + protected main).
- ☐ A CI pipeline (e.g. GitHub Actions) runs install, lint, and test steps on every push and pull request.
- ☐ The pipeline reports a clear pass/fail status visible on the pull request.
- ☐ A failing lint or test causes the pipeline to fail (verified with a deliberately broken commit on a scratch branch).

LEVEL 1 — DATA ENGINEERING — BRATS BRAIN-MRI PIPELINE

Tasks 9–20. Turn the raw BraTS dataset into clean, labelled, hospital-partitioned data that a Vision Transformer can train on. This level produces the inputs every model and federated round will consume.

Task 9 — Acquire and register BraTS dataset access

Obtain legitimate access to the BraTS Brain-MRI dataset and document the licence and usage terms.

Acceptance criteria (done when all are true):

- ☐ Dataset access is obtained through the official source and the licence/terms are saved in /docs.
- ☐ The raw dataset is stored in a documented location that is excluded from version control.
- ☐ A data-access note records who can use the data and under what conditions.
- ☐ The dataset version/year used is recorded for reproducibility.

Task 10 — Build a dataset download and verification script

Automate fetching and integrity-checking of the dataset so any environment can reproduce it.

Acceptance criteria (done when all are true):

- ☐ A script downloads or ingests the dataset into the expected folder layout.
- ☐ The script verifies file counts and checksums and reports any missing or corrupt files.
- ☐ Re-running the script is idempotent (does not re-download already-valid files).
- ☐ The script exits with a clear success/failure message and non-zero code on failure.

Task 11 — Inspect and document the BraTS structure

Explore the dataset's modalities (T1, T1ce, T2, FLAIR) and label conventions and write a short data dictionary.

Acceptance criteria (done when all are true):

- ☐ A notebook or report displays sample slices from each MRI modality.
- ☐ A data dictionary documents modalities, image dimensions, intensity ranges, and label meanings.
- ☐ The distribution of tumor classes / cases is summarised in a table or chart.
- ☐ Any anomalies (missing modalities, inconsistent shapes) are listed.

Task 12 — Implement the MRI volume loader

Write a loader that reads 3D MRI volumes (e.g. NIfTI) into arrays with correct orientation and spacing.

Acceptance criteria (done when all are true):

- ☐ The loader reads a volume and returns an array of the documented shape and dtype.
- ☐ Voxel spacing / affine information is preserved or recorded.
- ☐ The loader raises a clear error on unreadable or malformed files.
- ☐ A unit test loads a known sample and asserts its expected shape.

Task 13 — Extract 2D slices for classification

Convert 3D volumes into representative 2D slices suitable for a ViT image classifier.

Acceptance criteria (done when all are true):

- ☐ A configurable strategy selects slices (e.g. tumor-containing or central slices) from each volume.
- ☐ Extracted slices are saved or streamed with a consistent naming scheme linking them to source volume and class.
- ☐ The number of slices per class is logged to reveal imbalance.
- ☐ A visual spot-check confirms extracted slices contain the expected anatomy.

Task 14 — Build the image preprocessing pipeline

Standardise every image to the ViT input format: resize to 224x224, normalise intensities, and handle channels.

Acceptance criteria (done when all are true):

- ☐ Every image is resized to 224x224 (or the chosen ViT input size) and converted to the required channel format.
- ☐ Intensity normalisation is applied consistently and documented.
- ☐ The pipeline is deterministic given the same input (verified by re-running on one image).
- ☐ Output tensors have the exact shape and value range the ViT expects.

Task 15 — Implement the tumor-class labelling map

Map raw annotations to the target classification labels (e.g. glioma, meningioma, pituitary, and/or no-tumor).

Acceptance criteria (done when all are true):

- ☐ A single source-of-truth mapping converts raw labels to model class indices.
- ☐ The label set is documented and used identically in training and evaluation.
- ☐ Every sample resolves to exactly one valid label (no unmapped or null labels).
- ☐ A count per final class is produced to confirm the mapping.

Task 16 — Implement data augmentation

Add training-time augmentations appropriate for medical images to improve generalisation.

Acceptance criteria (done when all are true):

- ☐ Augmentations (e.g. flips, small rotations, intensity jitter) are applied only to the training split.
- ☐ Augmentations preserve medical validity (no transforms that would create anatomically impossible images).
- ☐ Augmentation is configurable and can be turned off for reproducible evaluation.
- ☐ Sample augmented images are visualised to confirm correctness.

Task 17 — Partition data across simulated hospitals

Split the dataset across N simulated hospital clients, deliberately introducing non-IID differences to mimic real-world domain shift.

Acceptance criteria (done when all are true):

- ☐ Data is divided into N (>=3) disjoint hospital partitions with no patient leakage across partitions.
- ☐ Partitions are non-IID (class or feature distribution differs between hospitals) and this difference is quantified.
- ☐ The partition assignment is reproducible from a fixed seed.
- ☐ A summary table shows sample and class counts per hospital.

Task 18 — Build PyTorch Dataset and DataLoader classes

Wrap the preprocessed, partitioned data in reusable Dataset/DataLoader classes for training.

Acceptance criteria (done when all are true):

- ☐ A Dataset class returns a correctly shaped (image, label) pair for any index.
- ☐ A DataLoader yields batches of the configured size with shuffling on the training split.
- ☐ Batches load without error for every hospital partition.
- ☐ A unit test iterates one full epoch of a small partition without crashing.

Task 19 — Create per-hospital train/val/test splits

Within each hospital partition, create reproducible training, validation, and test subsets.

Acceptance criteria (done when all are true):

- ☐ Each hospital partition is split into train/val/test with documented ratios.
- ☐ Splits are stratified by class where feasible and contain no overlap.
- ☐ The same seed reproduces identical splits.
- ☐ Split sizes per hospital are recorded in a table.

Task 20 — Produce a data-quality validation report

Generate an automated report confirming the data pipeline output is clean and ready for training.

Acceptance criteria (done when all are true):

- ☐ A report summarises total samples, per-hospital and per-class counts, and image shape consistency.
- ☐ The report flags any duplicates, corrupt images, or label mismatches.
- ☐ The report confirms no patient overlap between train/val/test.
- ☐ The report is regenerated by a single command and saved to /docs.

LEVEL 2 — VISION TRANSFORMER MODEL & LOCAL TRAINING

Tasks 21–34. Build and train the ViT-Base/16 classifier that each hospital will run locally. This is the 'brain' that federated learning will later coordinate. A single-hospital baseline here becomes the yardstick for measuring the federated system.

Task 21 — Load the ViT-Base/16 backbone

Bring in a Vision Transformer (ViT-Base/16) backbone, optionally with pretrained weights.

Acceptance criteria (done when all are true):

- ☐ A ViT-Base/16 model loads successfully (e.g. via timm) and accepts a 224x224 input batch.
- ☐ A forward pass on a dummy batch returns an output of the expected feature dimension.
- ☐ Whether pretrained weights are used is configurable and documented.
- ☐ The model's parameter count is printed and matches the expected magnitude for ViT-Base.

Task 22 — Add the tumor classification head

Attach a classification head sized to the tumor class set.

Acceptance criteria (done when all are true):

- ☐ The model outputs logits with length equal to the number of tumor classes.
- ☐ A forward pass followed by softmax produces a valid probability distribution summing to 1.
- ☐ The head is the only randomly initialised part when using a pretrained backbone.
- ☐ The full model runs end-to-end on one batch without shape errors.

Task 23 — Implement a model configuration and registry

Centralise model hyperparameters (input size, classes, dropout, etc.) so the exact same model can be rebuilt anywhere.

Acceptance criteria (done when all are true):

- ☐ A single config object/file fully specifies how to construct the model.
- ☐ A factory function builds the model from the config with no hidden defaults.
- ☐ Two builds from the same config produce architecturally identical models.
- ☐ The config is serialisable and stored alongside checkpoints.

Task 24 — Implement the loss function and class-imbalance handling

Choose and implement the training loss, accounting for class imbalance in the dataset.

Acceptance criteria (done when all are true):

- ☐ A loss function (e.g. weighted cross-entropy) is implemented and returns a scalar for a batch.
- ☐ Class weights or a sampling strategy address the imbalance measured in Level 1.
- ☐ The loss decreases on an intentional overfit test over a tiny batch.
- ☐ The imbalance-handling choice is documented with its rationale.

Task 25 — Implement the optimizer and learning-rate scheduler

Configure the optimizer and LR schedule suited to ViT fine-tuning.

Acceptance criteria (done when all are true):

- ☐ An optimizer (e.g. AdamW) and an LR scheduler are configured from the model config.
- ☐ Learning rate changes over epochs as the schedule intends (verified by logging).
- ☐ Hyperparameters are read from config, not hard-coded in the training loop.
- ☐ A short run shows stable (non-diverging) loss.

Task 26 — Build the local training loop

Implement a single-hospital training loop with train/validation phases per epoch.

Acceptance criteria (done when all are true):

- ☐ The loop trains for a configurable number of epochs and logs train and validation loss each epoch.
- ☐ Validation runs with gradients disabled and in eval mode.
- ☐ The loop is device-agnostic (runs on CPU, uses GPU when available).
- ☐ A short run completes end-to-end and produces a trained model object.

Task 27 — Implement evaluation metrics

Compute the classification metrics needed to judge medical model quality.

Acceptance criteria (done when all are true):

- ☐ Accuracy, precision, recall, F1 (per class and macro), and a confusion matrix are computed on the test split.
- ☐ ROC-AUC (or equivalent) is reported for the classification task.
- ☐ Metrics are computed by a reusable function that takes predictions and labels.
- ☐ Reported metrics match a manual calculation on a small known example.

Task 28 — Implement checkpointing

Save and restore model weights and training state reliably.

Acceptance criteria (done when all are true):

- ☐ Training saves checkpoints (weights + config + epoch) at a configurable interval and at best-validation.
- ☐ A saved checkpoint can be reloaded and reproduces the same validation metric.
- ☐ Checkpoint files are named to identify model, round/epoch, and metric.
- ☐ Loading a checkpoint into a fresh process resumes or evaluates correctly.

Task 29 — Implement experiment logging

Record metrics, hyperparameters, and curves for every run so results are traceable.

Acceptance criteria (done when all are true):

- ☐ Loss and metric curves are logged per epoch to a viewer (e.g. TensorBoard) or structured files.
- ☐ Each run records its full hyperparameter set.
- ☐ Two runs can be compared side by side from the logs.
- ☐ Logs are written to a documented location and excluded from version control.

Task 30 — Handle mixed precision and device placement

Add optional mixed-precision training and clean CPU/GPU handling.

Acceptance criteria (done when all are true):

- ☐ The model and batches move to the correct device automatically.
- ☐ Mixed precision can be enabled/disabled by config and runs without NaNs when enabled on GPU.
- ☐ The code runs correctly on a CPU-only machine (mixed precision gracefully disabled).
- ☐ A short GPU run shows reduced memory or faster steps versus full precision (if GPU available).

Task 31 — Train and record the single-hospital baseline

Train the ViT on one hospital's data and record the baseline metrics that federated learning must beat.

Acceptance criteria (done when all are true):

- ☐ A full training run on a single hospital completes and its test metrics are recorded in /docs.
- ☐ The baseline is reproducible from a fixed seed and stored config.
- ☐ The baseline result explicitly demonstrates the 'small-data' weakness (e.g. lower accuracy on other hospitals' data).
- ☐ The trained baseline checkpoint is saved for later comparison.

Task 32 — Serialise and deserialise model weights as a vector

Convert the model's weights to/from a flat numeric vector — the form federated learning and encryption will exchange.

Acceptance criteria (done when all are true):

- ☐ A function flattens the model state_dict into a 1-D vector of known length.
- ☐ An inverse function reloads a vector back into the model, restoring identical outputs.
- ☐ The round-trip (flatten → unflatten) leaves model predictions unchanged.
- ☐ The vector length and layer ordering are documented for the encryption layer.

Task 33 — Identify critical parameters for selective encryption

Determine which parts of the model (e.g. the [CLS] token / classification-critical parameters) are most important to encrypt, enabling the bandwidth saving.

Acceptance criteria (done when all are true):

- ☐ The specific parameters designated as 'critical' (e.g. [CLS] token related) are identified and documented.
- ☐ A function can extract just those critical parameters as a vector and reinsert them.
- ☐ The size ratio of critical parameters to the full model is measured and recorded.
- ☐ The rationale for this selection (accuracy vs bandwidth trade-off) is documented.

Task 34 — Extract model update deltas

Compute the update (delta) between the global model and a locally trained model — the quantity that gets shared each round.

Acceptance criteria (done when all are true):

- ☐ A function computes the element-wise difference between local and global weight vectors.
- ☐ Applying the delta back to the global model reconstructs the local model exactly.
- ☐ The delta can be restricted to only the critical parameters from Task 33.
- ☐ A unit test verifies delta + global == local within floating-point tolerance.

LEVEL 3 — HOMOMORPHIC ENCRYPTION — TENSEAL / CKKS

Tasks 35–46. Make it mathematically impossible for the central server to see any hospital's model updates. This level builds the encryption layer that locks updates before they leave a hospital and lets the server average them while still locked.

Task 35 — Install TenSEAL and validate CKKS basics

Install TenSEAL and confirm the CKKS scheme works with a minimal encrypt/compute/decrypt example.

Acceptance criteria (done when all are true):

- ☐ TenSEAL imports and a minimal CKKS example encrypts a small vector, adds a constant homomorphically, and decrypts the correct result.
- ☐ The CKKS scheme (not BFV) is confirmed as the one in use.
- ☐ A short written note explains, in plain language, what CKKS lets the project do.
- ☐ The example runs from a single command for any team member.

Task 36 — Create the CKKS encryption context

Build a reusable CKKS context with correctly chosen polynomial modulus degree, coefficient modulus sizes, and global scale.

Acceptance criteria (done when all are true):

- ☐ A context is created with documented poly_modulus_degree, coeff_mod_bit_sizes, and global_scale values.
- ☐ The chosen parameters support the required multiplicative depth for weighted averaging.
- ☐ The parameter choices and their security level are documented.
- ☐ Encrypting and decrypting a test vector through this context returns values within acceptable CKKS precision.

Task 37 — Implement secure key generation and management

Generate and manage the CKKS keys (secret, public, Galois/relin) with clear ownership rules.

Acceptance criteria (done when all are true):

- ☐ Key generation produces the secret key plus the public/Galois/relin keys needed for the planned operations.
- ☐ Keys are stored/loaded through a dedicated key-management module.
- ☐ It is documented and enforced which keys stay private to a hospital and which may be shared.
- ☐ Regenerating keys from the same seed/context behaves consistently and is testable.

Task 38 — Encrypt a model-weight vector

Encrypt a flattened weight (or delta) vector into a CKKS ciphertext.

Acceptance criteria (done when all are true):

- ☐ A weight vector is encrypted into a CKKS vector without error.
- ☐ The ciphertext is produced using only the public key / context (no secret key needed to encrypt).
- ☐ Encrypting the same vector twice yields ciphertexts that both decrypt to the original.
- ☐ The operation handles the real vector length from Task 32 (or a documented chunking strategy).

Task 39 — Implement decryption with correctness verification

Decrypt ciphertexts and verify the encrypt→decrypt round-trip is accurate enough for training.

Acceptance criteria (done when all are true):

- ☐ Decryption with the secret key recovers the original vector within a documented tolerance.
- ☐ A test asserts the mean/max reconstruction error is below an acceptable CKKS threshold.
- ☐ Decryption fails or is impossible without the correct secret key (verified).
- ☐ The precision loss is quantified and confirmed not to harm model accuracy materially.

Task 40 — Implement selective encryption of critical parameters

Encrypt only the critical parameters identified in Task 33, sending the rest in a cheaper form, to cut bandwidth.

Acceptance criteria (done when all are true):

- ☐ Only the designated critical parameters are encrypted; the split is driven by the Task 33 selection.
- ☐ The encrypted-plus-remaining payload can be reassembled into a full update on receipt.
- ☐ The payload size is measurably smaller than encrypting the entire model.
- ☐ Model accuracy with selective encryption is confirmed comparable to full encryption on a small test.

Task 41 — Implement homomorphic addition of encrypted vectors

Add two or more encrypted update vectors together without decrypting — the core of aggregation.

Acceptance criteria (done when all are true):

- ☐ Adding two ciphertexts and decrypting the result equals the sum of the two plaintext vectors (within tolerance).
- ☐ Addition works across the number of hospitals expected in a round.
- ☐ The operation requires no secret key.
- ☐ A unit test verifies homomorphic sum against a plaintext sum.

Task 42 — Implement homomorphic scalar multiplication

Multiply an encrypted vector by a plaintext scalar (e.g. a client's sample-weight) without decrypting — needed for weighted averaging.

Acceptance criteria (done when all are true):

- ☐ Multiplying a ciphertext by a plaintext scalar and decrypting equals scalar × plaintext vector (within tolerance).
- ☐ Combined with addition, a weighted average of encrypted vectors decrypts to the correct weighted average.
- ☐ The operation stays within the multiplicative depth budgeted in the context.
- ☐ A unit test verifies an encrypted weighted average against the plaintext equivalent.

Task 43 — Serialise encrypted tensors for network transfer

Convert ciphertexts to/from bytes so they can be sent over the network to the server.

Acceptance criteria (done when all are true):

- ☐ A ciphertext serialises to bytes and deserialises back to a usable ciphertext.
- ☐ A deserialised ciphertext still decrypts (or aggregates) to the correct result.
- ☐ The serialised size is measured and logged.
- ☐ Serialisation excludes the secret key so transferred bytes cannot be decrypted by a third party.

Task 44 — Benchmark encryption size, time, and bandwidth savings

Measure the real cost of encryption and confirm the selective-encryption bandwidth reduction claim.

Acceptance criteria (done when all are true):

- ☐ Encryption/decryption time per update is measured and recorded.
- ☐ Payload size for full vs selective encryption is compared, and the reduction factor is reported.
- ☐ The measured bandwidth reduction is stated against the project's target (up to ~30x).
- ☐ Results are saved to /docs with the hardware used.

Task 45 — Enforce private-key isolation

Guarantee, in code and design, that a hospital's secret key never leaves that hospital.

Acceptance criteria (done when all are true):

- ☐ The server-side code never has access to any secret key (verified by inspection/tests).
- ☐ Only public keys/contexts and ciphertexts cross the network from client to server.
- ☐ An automated check fails if a secret key would be serialised into an outbound message.
- ☐ The key-isolation design is documented in the architecture notes.

Task 46 — Security-test that the server cannot read plaintext

Prove that from what the server holds, no plaintext model update or patient signal can be recovered.

Acceptance criteria (done when all are true):

- ☐ A test confirms the server, with only its available keys, cannot decrypt any client ciphertext.
- ☐ Attempting decryption server-side raises an error or yields garbage, and this is asserted in a test.
- ☐ A written threat-model note explains what an attacker at the server can and cannot see.
- ☐ The test is part of the automated suite so regressions are caught.

LEVEL 4 — FEDERATED LEARNING ORCHESTRATION — FLOWER + ENCRYPTED FEDAVG

Tasks 47–60. Connect the pieces: multiple hospital clients train locally, encrypt their updates, and a server averages them while encrypted, then returns an improved global model. This is the heart of MedShield-FL.

Task 47 — Set up the Flower server and client scaffolding

Create the basic Flower server and client processes that can connect and complete an empty round.

Acceptance criteria (done when all are true):

- ☐ A Flower server starts and at least one client connects to it successfully.
- ☐ A trivial round (no real training) completes end-to-end without connection errors.
- ☐ Server and client addresses/ports are configurable via environment variables.
- ☐ Logs show the round starting and finishing on both server and client.

Task 48 — Implement the custom Flower client (fit/evaluate)

Wrap the Level-2 local ViT training inside a Flower client's fit() and evaluate() methods.

Acceptance criteria (done when all are true):

- ☐ The client's fit() trains the local ViT for the configured local epochs and returns updated parameters and sample count.
- ☐ The client's evaluate() returns real validation loss and metrics on local data.
- ☐ The client receives global parameters, loads them into the model, and trains from them.
- ☐ One client completes a real fit/evaluate cycle on its hospital partition.

Task 49 — Add client-side encryption inside fit()

Encrypt the model update inside the client before it returns parameters to the server.

Acceptance criteria (done when all are true):

- ☐ fit() returns encrypted (and/or selectively encrypted) parameters, never plaintext weights.
- ☐ The client uses the Level-3 encryption layer with its local keys.
- ☐ The returned payload is serialised bytes ready for transport.
- ☐ A test confirms no plaintext weight vector leaves the client.

Task 50 — Implement the custom encrypted-FedAvg strategy

Write a Flower Strategy that aggregates encrypted client updates instead of plaintext ones.

Acceptance criteria (done when all are true):

- ☐ A custom Strategy subclass overrides aggregation to operate on encrypted vectors.
- ☐ The strategy assembles all clients' ciphertexts for a round without decrypting them.
- ☐ The strategy configures each round (which clients, local epochs, etc.).
- ☐ The strategy integrates cleanly with the Flower server from Task 47.

Task 51 — Implement encrypted homomorphic aggregation

On the server, compute the weighted average of encrypted updates using homomorphic add and scalar-multiply.

Acceptance criteria (done when all are true):

- ☐ The server produces an aggregated ciphertext using only homomorphic operations (no decryption).
- ☐ Aggregation weights each client by its sample count (FedAvg) via homomorphic scalar multiplication.
- ☐ The aggregated ciphertext, when decrypted by a client, equals the correct plaintext weighted average within tolerance.
- ☐ Aggregation succeeds for the full number of participating hospitals.

Task 52 — Implement round orchestration

Drive the full round cycle: distribute the global model, collect encrypted updates, aggregate, and store the new global model.

Acceptance criteria (done when all are true):

- ☐ A round distributes the current global model to all selected clients.
- ☐ The server collects every client's encrypted update before aggregating.
- ☐ The aggregated result becomes the new global model for the next round.
- ☐ A configurable number of rounds runs back-to-back without manual intervention.

Task 53 — Implement client-side decryption of the global update

Each client decrypts the returned aggregated model locally with its own secret key.

Acceptance criteria (done when all are true):

- ☐ After a round, the client decrypts the aggregated update and applies it to its local model.
- ☐ Decryption uses only the client's local secret key.
- ☐ The decrypted global model produces sensible predictions (no corruption).
- ☐ A test confirms the applied global model differs from the pre-round model as expected.

Task 54 — Handle sample-count weighting for FedAvg

Ensure aggregation correctly weights hospitals by how much data they contributed.

Acceptance criteria (done when all are true):

- ☐ Each client reports its training sample count to the server each round.
- ☐ The homomorphic weighted average uses these counts so larger hospitals count proportionally more.
- ☐ A plaintext simulation confirms the encrypted weighted average matches the intended FedAvg formula.
- ☐ Edge cases (a hospital with zero new samples) are handled without breaking a round.

Task 55 — Run a multi-client simulation

Simulate three or more hospitals training together against the server.

Acceptance criteria (done when all are true):

- ☐ >=3 hospital clients participate in the same federated run using their non-IID partitions.
- ☐ All clients complete each round and the server aggregates all of their updates.
- ☐ The run is reproducible from configuration and seeds.
- ☐ Per-client and global metrics are logged for every round.

Task 56 — Implement client dropout and fault tolerance

Keep training working when a hospital disconnects or fails mid-round.

Acceptance criteria (done when all are true):

- ☐ If a client fails to report in a round, the server still aggregates the remaining clients and continues.
- ☐ A disconnected client can rejoin in a later round and receive the current global model.
- ☐ The system logs which clients participated in each round.
- ☐ A simulated mid-round dropout does not crash the server or corrupt the global model.

Task 57 — Secure the client-server communication channel

Encrypt the transport layer (in addition to the homomorphic layer) between clients and server.

Acceptance criteria (done when all are true):

- ☐ Client-server communication uses TLS/secure gRPC, configurable via settings.
- ☐ A connection without valid transport security is refused or clearly flagged.
- ☐ Transport security is documented as a second, independent layer from homomorphic encryption.
- ☐ A round completes successfully over the secured channel.

Task 58 — Persist per-round metrics and global model versions

Store every round's metrics and a versioned global model so progress is auditable.

Acceptance criteria (done when all are true):

- ☐ Each round's accuracy, loss, and participating hospitals are written to the database.
- ☐ Each new global model version is stored and retrievable by round number.
- ☐ The history can be queried to show accuracy improving over rounds.
- ☐ Stored model versions can be reloaded and evaluated later.

Task 59 — Run full federated training to convergence

Execute a complete multi-round federated training run and confirm the global model improves and stabilises.

Acceptance criteria (done when all are true):

- ☐ A full run of many rounds completes and global validation accuracy improves over the early rounds.
- ☐ Accuracy curves over rounds are saved and show convergence (plateauing) behaviour.
- ☐ The final global model is checkpointed and versioned.
- ☐ The run completes without decryption ever happening on the server.

Task 60 — Compare federated vs centralized vs single-hospital

Quantify the benefit of MedShield-FL by comparing against baselines.

Acceptance criteria (done when all are true):

- ☐ Final metrics are reported for single-hospital (Task 31), centralized (all data pooled), and federated models.
- ☐ The federated model beats the single-hospital baseline on cross-hospital test data.
- ☐ The accuracy gap between federated and centralized is measured and discussed.
- ☐ A comparison table/chart is saved to /docs.

LEVEL 5 — ACTIVE LEARNING — SMART LABELLING ASSISTANT

Tasks 61–68. Save doctors' scarce time by having the system pick only the most useful, uncertain images for them to label, instead of labelling everything. This raises accuracy per unit of expert effort.

Task 61 — Implement uncertainty estimation

Measure how unsure the model is about each unlabeled image.

Acceptance criteria (done when all are true):

- ☐ An uncertainty score (e.g. prediction entropy, least-confidence, or MC-dropout variance) is computed per image.
- ☐ Higher scores demonstrably correspond to harder/ambiguous images on a spot-check.
- ☐ The method is configurable so alternative strategies can be swapped in.
- ☐ Scores are reproducible for the same model and image.

Task 62 — Manage the unlabeled data pool per hospital

Track which local images are labelled vs unlabeled at each hospital.

Acceptance criteria (done when all are true):

- ☐ Each hospital maintains a pool of unlabeled images distinct from its labelled set.
- ☐ Newly labelled images move from the unlabeled pool to the labelled set automatically.
- ☐ The pool state is persisted so it survives a restart.
- ☐ Pool sizes (labelled vs unlabeled) are queryable.

Task 63 — Implement the query strategy

Select the most informative unlabeled images for labelling based on uncertainty (and optionally diversity).

Acceptance criteria (done when all are true):

- ☐ The strategy ranks unlabeled images and returns the top-K most informative ones.
- ☐ The selection avoids near-duplicate images where a diversity criterion is enabled.
- ☐ The selection is deterministic given the same model and pool.
- ☐ The chosen images visibly skew toward uncertain cases versus a random draw.

Task 64 — Implement labelling budget and batch selection

Respect a configurable labelling budget per round so doctors are not overloaded.

Acceptance criteria (done when all are true):

- ☐ A configurable budget limits how many images are queued for labelling per round/hospital.
- ☐ The system never queues more than the budget allows.
- ☐ Remaining budget is tracked and reported.
- ☐ Changing the budget in config changes the batch size with no code edit.

Task 65 — Build the labelling-queue surface

Provide the data/API that presents flagged images to a doctor for review.

Acceptance criteria (done when all are true):

- ☐ An endpoint/service returns the current queue of images awaiting labels for a hospital.
- ☐ Each queued item includes the image reference and the model's current (uncertain) prediction.
- ☐ The queue reflects the query strategy's ordering (most useful first).
- ☐ The queue updates after images are labelled.

Task 66 — Implement label submission and dataset update

Let a doctor's submitted label flow back into the training data.

Acceptance criteria (done when all are true):

- ☐ A submitted label is validated against the allowed class set and stored.
- ☐ The newly labelled image is added to the hospital's labelled training set.
- ☐ Submitting a label removes that image from the pending queue.
- ☐ An audit record captures who labelled what and when.

Task 67 — Integrate active learning into the federated loop

Close the loop: newly labelled data is used in subsequent federated rounds.

Acceptance criteria (done when all are true):

- ☐ Between federated rounds, newly labelled images are included in the next local training.
- ☐ The pipeline runs multiple query→label→train cycles without manual data handling.
- ☐ The growing labelled-set size per hospital is logged over cycles.
- ☐ A full cycle (select → label → federated round) completes end-to-end.

Task 68 — Evaluate active learning versus random sampling

Prove active learning reaches higher accuracy with fewer labels.

Acceptance criteria (done when all are true):

- ☐ Two runs — active-learning selection vs random selection — are compared at equal labelling budgets.
- ☐ Active learning reaches a target accuracy with fewer labelled images, and the saving is quantified.
- ☐ A learning curve (accuracy vs number of labels) is plotted for both.
- ☐ Results are saved to /docs with the experimental setup.

LEVEL 6 — EXPLAINABLE AI — GRAD-CAM FOR VIT

Tasks 69–76. Give doctors a reason to trust the AI: a heatmap showing which part of the scan drove each prediction, plus a plain-language note. Without this, clinicians will not accept a 'black-box' decision.

Task 69 — Implement Grad-CAM adapted for Vision Transformers

Implement a Grad-CAM (or attention-rollout style) method that works with the ViT architecture, not just CNNs.

Acceptance criteria (done when all are true):

- ☐ A Grad-CAM variant appropriate for ViT is implemented and produces a saliency map for a prediction.
- ☐ The method targets a suitable ViT layer/attention block, documented with rationale.
- ☐ The map has the same spatial extent as the input image after upsampling.
- ☐ Running it on a clear tumor case highlights a plausible region on inspection.

Task 70 — Hook into ViT layers to capture gradients and activations

Register the forward/backward hooks needed to capture the activations and gradients Grad-CAM requires.

Acceptance criteria (done when all are true):

- ☐ Hooks capture activations and gradients from the chosen layer during a single forward/backward pass.
- ☐ Hooks are cleanly removed after use (no memory leaks across many calls).
- ☐ The captured tensors have the expected shapes.
- ☐ Capturing works in inference mode without disturbing normal predictions.

Task 71 — Generate the heatmap for a prediction

Turn captured signals into a normalised heatmap for the predicted class.

Acceptance criteria (done when all are true):

- ☐ A heatmap is produced for the model's predicted class (and optionally any chosen class).
- ☐ Heatmap values are normalised to a consistent range for display.
- ☐ Generation works for any valid input image without error.
- ☐ The heatmap changes meaningfully between different input images.

Task 72 — Overlay the heatmap on the original MRI slice

Composite the heatmap over the original scan so the highlighted region is visible in context.

Acceptance criteria (done when all are true):

- ☐ An overlay image shows the original MRI with the heatmap superimposed at correct alignment.
- ☐ Overlay opacity/colormap is configurable and legible on greyscale MRI.
- ☐ The overlay is exportable as an image file.
- ☐ Alignment is verified so the highlight sits over the correct anatomy.

Task 73 — Generate a plain-language explanation

Accompany each heatmap with a short, non-technical sentence a doctor can read quickly.

Acceptance criteria (done when all are true):

- ☐ Each prediction yields a short plain-language explanation (predicted class, confidence, and where the model looked).
- ☐ The wording avoids jargon and states confidence clearly.
- ☐ The explanation is generated automatically from the prediction and heatmap, not hand-written.
- ☐ The explanation is consistent with the displayed heatmap and prediction.

Task 74 — Expose explainability via an API endpoint

Provide an endpoint that returns the prediction, heatmap overlay, and explanation for an uploaded scan.

Acceptance criteria (done when all are true):

- ☐ An endpoint accepts an image and returns the prediction, confidence, heatmap overlay, and explanation text.
- ☐ The endpoint handles invalid input with a clear error response.
- ☐ Response time for a single image is within a documented acceptable limit.
- ☐ The endpoint is covered by an automated test using a sample image.

Task 75 — Store and cache explanation artifacts

Persist generated heatmaps/explanations so they need not be recomputed and can be reviewed later.

Acceptance criteria (done when all are true):

- ☐ Generated overlays and explanations are stored and linked to their prediction record.
- ☐ Re-requesting an explanation for the same prediction returns the cached artifact.
- ☐ Stored artifacts are retrievable by prediction ID.
- ☐ Storage location and retention are documented.

Task 76 — Validate explanations qualitatively

Sanity-check that highlighted regions align with actual tumor locations.

Acceptance criteria (done when all are true):

- ☐ On a set of known-tumor cases, heatmaps are compared against the true tumor regions.
- ☐ A qualitative review notes agreement rate and any systematic failures.
- ☐ At least a handful of example overlays are saved to /docs as evidence.
- ☐ Findings and limitations are written up for the clinical-trust narrative.

LEVEL 7 — BACKEND SERVICES & DATABASE

Tasks 77–86. Wrap everything in a secure FastAPI service backed by PostgreSQL: authentication, hospitals, rounds, predictions, labelling, and audit trails. This is the API the dashboard and hospitals talk to.

Task 77 — Design the PostgreSQL schema

Model the entities the system needs: hospitals, users, training rounds, model versions, images, labels, predictions, and audit logs.

Acceptance criteria (done when all are true):

- ☐ An entity-relationship diagram documents all tables and their relationships.
- ☐ The schema captures hospitals, users/roles, rounds, model versions, images, labels, predictions, and audit records.
- ☐ Foreign keys and constraints enforce referential integrity.
- ☐ The schema deliberately stores no raw patient images centrally (only references/metadata), consistent with the privacy design.

Task 78 — Implement database models and migrations

Create ORM models and versioned migrations matching the schema.

Acceptance criteria (done when all are true):

- ☐ SQLAlchemy (or equivalent) models exist for every table in the schema.
- ☐ Alembic migrations create the full schema on an empty database.
- ☐ Migrations run forward and backward cleanly.
- ☐ A seed script inserts sample hospitals/users for development.

Task 79 — Implement the FastAPI application skeleton

Set up the API app with routing, dependency injection, error handling, and OpenAPI docs.

Acceptance criteria (done when all are true):

- ☐ The FastAPI app starts and serves interactive OpenAPI docs.
- ☐ Routers are organised by domain (auth, hospitals, rounds, predictions, labelling, metrics).
- ☐ A global error handler returns consistent JSON error responses.
- ☐ A health-check endpoint returns service and database status.

Task 80 — Implement authentication and authorization

Add JWT-based auth with roles for admin, doctor, and hospital operator.

Acceptance criteria (done when all are true):

- ☐ Users authenticate and receive a signed JWT; protected endpoints reject requests without a valid token.
- ☐ Roles (admin/doctor/hospital) gate access so each role can only reach permitted endpoints.
- ☐ Passwords are stored hashed, never in plaintext.
- ☐ Token expiry and refresh behaviour are implemented and tested.

Task 81 — Implement hospital and training-round endpoints

Expose endpoints to register hospitals and to read/manage federated training rounds.

Acceptance criteria (done when all are true):

- ☐ Endpoints list and register hospitals and return their participation status.
- ☐ Endpoints return training-round history with per-round accuracy and participants.
- ☐ Only authorized roles can trigger or view round management actions.
- ☐ Endpoints are covered by automated tests against a test database.

Task 82 — Implement the prediction endpoint

Let an authorized user upload an MRI and receive a tumor-class prediction from the current global model.

Acceptance criteria (done when all are true):

- ☐ An endpoint accepts an MRI image and returns the predicted class with confidence.
- ☐ The prediction uses the current global model version and records which version was used.
- ☐ Invalid or non-image uploads are rejected with a clear error.
- ☐ Each prediction is stored with a timestamp and user reference.

Task 83 — Implement the explainability endpoint integration

Wire the Grad-CAM service (Level 6) into the backend so predictions can return explanations.

Acceptance criteria (done when all are true):

- ☐ An endpoint returns the heatmap overlay and plain-language explanation for a given prediction.
- ☐ The explanation is linked to the stored prediction record.
- ☐ The endpoint reuses cached artifacts where available (Task 75).
- ☐ Access is restricted to authorized roles and tested.

Task 84 — Implement doctor labelling-queue endpoints

Expose the active-learning queue and label submission to doctors through the API.

Acceptance criteria (done when all are true):

- ☐ An endpoint returns a doctor's pending labelling queue for their hospital.
- ☐ An endpoint accepts a submitted label, validates it, and updates the dataset (Task 66).
- ☐ Only the doctor role can access these endpoints.
- ☐ Submitting a label is reflected immediately in the queue and audit log.

Task 85 — Implement metrics and monitoring endpoints

Provide the data the dashboard needs: round history, accuracy trend, and hospital participation.

Acceptance criteria (done when all are true):

- ☐ Endpoints return accuracy-over-rounds, current round, and per-hospital contribution counts.
- ☐ Data is queried efficiently (no full-table scans on large tables).
- ☐ Responses are shaped for direct use by the frontend charts.
- ☐ Endpoints are covered by tests with seeded data.

Task 86 — Implement audit logging and privacy-status tracking

Record security-relevant events and expose a verifiable 'data stayed local' privacy status.

Acceptance criteria (done when all are true):

- ☐ Key actions (logins, label submissions, predictions, round events) are written to an audit log.
- ☐ A privacy-status endpoint reports that no raw patient data left any hospital, backed by the encryption/isolation design.
- ☐ Audit records are immutable/append-only and timestamped.
- ☐ The privacy status reflects real system state, not a hard-coded value.

LEVEL 8 — FRONTEND DASHBOARD — REACT + TYPESCRIPT + TAILWIND

Tasks 87–96. Build the clean, non-technical dashboard hospital staff, admins, and doctors actually use. It only ever shows information — it never touches raw patient data or encryption keys.

Task 87 — Set up the app shell, routing, and theme

Create the overall layout, navigation, and a clean clinical Tailwind theme.

Acceptance criteria (done when all are true):

- ☐ A responsive app shell with navigation and page routing renders for all main screens.
- ☐ A consistent clean, high-contrast clinical theme is applied via Tailwind.
- ☐ Routing guards send unauthenticated users to the login screen.
- ☐ The layout works on desktop and tablet widths.

Task 88 — Implement the API client and auth flow

Build the typed API client and the login/logout/token handling.

Acceptance criteria (done when all are true):

- ☐ A typed API client wraps backend calls and injects the auth token automatically.
- ☐ Login stores the token in memory/state and logout clears it (no insecure persistent storage of secrets).
- ☐ Expired-token responses redirect the user to log in again.
- ☐ API and network errors surface as user-friendly messages.

Task 89 — Build the hospital dashboard

Show the current training round, model accuracy, and this hospital's contribution.

Acceptance criteria (done when all are true):

- ☐ The dashboard displays current round number, latest global accuracy, and images contributed by this hospital.
- ☐ Values are fetched live from the metrics endpoints.
- ☐ Loading and empty states are handled gracefully.
- ☐ The numbers shown match the backend data for a seeded scenario.

Task 90 — Build the admin panel

Show connected hospitals, encryption status, and overall system health.

Acceptance criteria (done when all are true):

- ☐ The admin panel lists connected hospitals and their participation/encryption status.
- ☐ Overall system health (services up, DB reachable) is displayed.
- ☐ Only admin-role users can open this panel.
- ☐ Status reflects real backend state and updates on refresh.

Task 91 — Build the doctor's labelling queue

Let doctors view flagged images, zoom/annotate, and submit labels.

Acceptance criteria (done when all are true):

- ☐ The queue shows the active-learning flagged images with the model's current prediction.
- ☐ A doctor can zoom/inspect an image and select a label from the allowed classes.
- ☐ Submitting a label calls the backend and removes the item from the queue.
- ☐ The screen is usable without technical knowledge and shows clear confirmation on submit.

Task 92 — Build the prediction screen

Let a user upload an MRI and see the predicted tumor class and confidence.

Acceptance criteria (done when all are true):

- ☐ A user can upload an MRI image and receive a predicted class with a confidence value.
- ☐ Upload validation rejects unsupported files with a clear message.
- ☐ A loading indicator shows while the prediction is computed.
- ☐ The result view links through to the explainability screen.

Task 93 — Build the explainability screen

Show the original scan, the Grad-CAM heatmap overlay, and the plain-language explanation.

Acceptance criteria (done when all are true):

- ☐ The screen displays the original MRI, the heatmap overlay, and the plain-language explanation together.
- ☐ The overlay is clearly legible over the greyscale scan.
- ☐ The explanation and confidence are shown in plain language.
- ☐ The data is fetched from the explainability endpoint for the selected prediction.

Task 94 — Build the privacy-status indicator

Give users a visible, trustworthy confirmation that data never left the hospital.

Acceptance criteria (done when all are true):

- ☐ A visible indicator (e.g. “Data stayed local ✓”) is shown, driven by the backend privacy-status endpoint.
- ☐ The indicator reflects real state and is not a static label.
- ☐ Hovering/opening it reveals a short plain explanation of what it means.
- ☐ The indicator is present on the main dashboard.

Task 95 — Build training-progress charts

Visualise accuracy over rounds and hospital participation over time.

Acceptance criteria (done when all are true):

- ☐ A chart shows global accuracy improving across training rounds.
- ☐ A view shows how many hospitals participated per round.
- ☐ Charts read cleanly in the black-and-white/clinical theme and are labelled.
- ☐ Chart data matches the metrics endpoints for a seeded scenario.

Task 96 — Polish responsiveness, states, and accessibility

Ensure the whole app handles loading/error states well and meets basic accessibility.

Acceptance criteria (done when all are true):

- ☐ Every screen has defined loading, empty, and error states.
- ☐ The app is keyboard-navigable and meets basic contrast/ARIA accessibility checks.
- ☐ Layouts adapt cleanly across desktop and tablet.
- ☐ No console errors occur during a normal walkthrough of all screens.

LEVEL 9 — INTEGRATION, TESTING, SECURITY & DEPLOYMENT

Tasks 97–100. Prove the whole system works together, is secure and private by design, performs acceptably, and can be deployed and demonstrated end-to-end.

Task 97 — Run an end-to-end integration test

Exercise the full pipeline in one flow: local training → encryption → aggregation → decryption → prediction → explanation → dashboard.

Acceptance criteria (done when all are true):

- ☐ An automated end-to-end scenario runs a federated round and then a prediction with explanation surfaced on the dashboard.
- ☐ The flow completes with no manual steps and no server-side decryption.
- ☐ The dashboard reflects the round's updated accuracy after the flow.
- ☐ The test is repeatable in CI against the containerised stack.

Task 98 — Build the comprehensive test and security suite

Cover the system with unit, integration, and dedicated privacy/security tests.

Acceptance criteria (done when all are true):

- ☐ Unit and integration tests cover data, model, encryption, FL, and API layers with a documented coverage target.
- ☐ Security tests assert private keys never leave clients and the server cannot decrypt updates (Tasks 45–46).
- ☐ Auth/role tests confirm unauthorized access is blocked on every protected endpoint.
- ☐ The full suite runs green in CI.

Task 99 — Benchmark performance and write documentation

Measure end-to-end performance and produce the documentation and diagrams the project needs.

Acceptance criteria (done when all are true):

- ☐ Performance is benchmarked (round time, encryption overhead, bandwidth, prediction latency) and recorded.
- ☐ A README, API documentation, and an architecture diagram are complete and accurate.
- ☐ A results section reports federated vs baseline accuracy and the bandwidth-saving figure.
- ☐ Setup instructions let a new user run the full system from a clean clone.

Task 100 — Deploy, demo, and validate compliance

Deploy the system (via Docker Compose / production config), prepare a demo, and validate the privacy narrative.

Acceptance criteria (done when all are true):

- ☐ The full stack deploys with a single documented command and runs a scripted demo with sample hospitals.
- ☐ A demo walkthrough shows collaborative training, prediction, explanation, and the privacy indicator.
- ☐ A compliance checklist maps system features to HIPAA/GDPR privacy principles (data never centralised, encrypted updates, audit logs).
- ☐ A final validation confirms all objectives from the project document are met, with evidence linked in /docs.

Part E · Glossary of Key Terms

Term	Meaning
Federated Learning (FL)	A way to train one shared model across many devices/organisations where the data stays local and only model updates are shared.
FedAvg	Federated Averaging — the algorithm that combines hospitals' updates into a new global model, weighted by how much data each contributed.
Homomorphic Encryption (HE)	Encryption that allows computation (e.g. addition) directly on encrypted data, so results stay correct without ever decrypting.
CKKS	A homomorphic encryption scheme (implemented here via TenSEAL) that works on real numbers — ideal for encrypting model weights.
TenSEAL	A Python library providing CKKS homomorphic encryption on tensors.
Vision Transformer (ViT)	An image model that splits an image into patches and processes them like tokens; ViT-Base/16 uses 16×16 patches.
[CLS] token	A special summary token in a transformer whose final state represents the whole image — a compact, critical part to encrypt.
Grad-CAM	Explainable-AI method producing a heatmap of the image regions most responsible for a prediction.
Active Learning	Selecting the most informative (uncertain) samples for expert labelling, to get more accuracy from less labelling effort.
Domain shift	When data differs between sources (e.g. different scanners/hospitals), which can hurt a model trained on only one source.
Non-IID data	Data that is not identically distributed across clients — realistic for hospitals and a core FL challenge.
HIPAA / GDPR	US and EU privacy regulations that restrict sharing of patient data — the reason federated learning is needed here.
BraTS	Brain Tumor Segmentation benchmark: multi-hospital, expert-labelled brain-MRI scans (T1, T1ce, T2, FLAIR).